

AI Agent Harness 執行框架

微服务拆分实施方案

Django 单体 → Django(业务) + FastAPI(编排) 双服务架构

v1.0 | 2025年7月

项目	AI Agent Harness 执行框架
当前架构	Django 单体 (业务 + Agent编排一体化)
目标架构	Django(业务) + FastAPI(AI编排) + PostgreSQL + Redis + Nginx
部署方案	Docker Compose 自建微服务 (ECS 2核8G)
拆分周期	2周 (7月17日 → 8月6日)
核心目标	AI编排层独立部署, 实现业务层与AI层解耦

目录

- 一. 当前状态: 有什么, 长什么样
- 二. 为什么拆: 问题在哪里
- 三. 目标架构: 拆完是什么样
- 四. 拆分方案: 文件级迁移清单 (最硬核部分)
- 五. Step by Step: 7天执行清单
- 六. Docker Compose 完整配置
- 七. Django 侧改动: 从直接调用到 HTTP 请求
- 八. 第二周: 加固与交付物
- 九. 面试话术对照 (拆完后怎么讲)
- 附录: 完整文件结构对比

一. 当前状态: 有什么, 长什么样

当前项目是 Django REST Framework 单体应用, 前端 Vue 3. 所有代码在一个大箱子里, 包括业务功能和 AI 编排引擎.

1.1 当前目录结构 (关键部分)

```
backend/ |-- agent_gateway/ # 业务入口: Django View + Celery Task | |-- views.py # [关键] import build_default_workflow,
run_workflow_stream | |-- tasks.py # Celery 异步任务 (也直接 import Agent) | |-- core/ | |-- agents/ # * 8个文件: 要搬走的 "AI编排核心"
| |-- harness/ | | |-- state.py # LangGraph AgentState 定义 | | |-- workflow.py # build_default_workflow, run_workflow_stream | |
|-- self_healing/engine.py # 自愈引擎 | |-- base_agent.py # Agent 基类 | |-- plan_agent.py # 计划 Agent | |--
testcase_gen_agent.py | |-- data_factory_agent.py | |-- execution_agent.py | |-- evaluator_agent.py | |-- knowledge_agent.py | |
|-- tools/ # 工具层: 也要搬走 | |-- knowledge_search.py | |-- milvus_store.py | |-- testcase_storage.py | |-- ... (共9个文件) | | |--
mcp/ # MCP 协议层: 也要搬走 | |-- client.py, server.py, tools_adapter.py, transport.py | | |-- models/ | |-- router.py # LLM 路由:
也要搬走 | |-- prompts/ # Prompt 模板: 也要搬走
```

1.2 Django 如何调用 Agent 编排 (关键入口)

Django 的 agent_gateway/views.py 是整个 AI 功能的入口, 它直接 import Agent 代码:

```
# agent_gateway/views.py (当前写法) from core.agents.harness.workflow import build_default_workflow, run_workflow_stream from
core.agents.testcase_gen_agent import TestCaseGeneratorAgent from core.agents.data_factory_agent import DataFactoryAgent
from core.agents.execution_agent import ExecutionEngineAgent from core.agents.evaluator_agent import EvaluatorAgent # ->
函数调用方式, Agent 和 Django 在同一个 Python 进程中
```

[现状总结] 所有 Agent 代码在 Django 进程内运行. Django 崩了 -> Agent 也崩了. Agent 想独立扩容 -> 做不到.

二. 为什么拆: 问题在哪里

耦合太紧

Agent 代码和 Django 在同一进程. Django 项目里任何一处改动 (改了 settings, 换了中间件), 都可能影响 Agent 执行. 反之亦然 -- Agent 调 LLM 阻塞了 30 秒, Django 的一个 Worker 就被占住 30 秒.

无法独立扩容

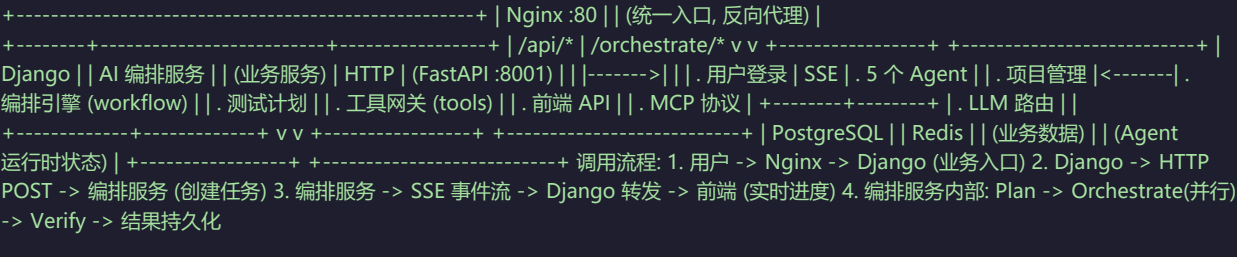
你要加 Agent 运算能力? 只能加 Django Worker (带着全套 ORM, 中间件, 模板引擎一起扩). 效率极低. 拆开后, Agent 服务可以单独扩 5 个实例, Django 保持不变.

面试说服力不够

面试官看到 Django 项目 四个字, 脑子里浮现的是一个 Web CRUD 后端. 你得花 10 分钟解释它里面有一套 Agent 编排引擎. 拆开以后, 简历上直接写独立 AI 编排微服务 -- 他自己能想象.

三. 目标架构: 拆完是什么样

3.1 架构图 (拆完后的服务拓扑)



3.2 对比总结

维度	拆之前	拆之后
Agent 运行位置	和 Django 同一进程	独立的 FastAPI 进程
Agent 扩容方式	多开 Django Worker (带全部ORM)	单独 docker-compose scale
Django 崩溃影响	Agent 同时挂	Agent 不受影响
Agent 崩溃影响	Django Worker 被占用	Django 无感知
Django 调用 Agent	直接 import 函数调用	HTTP POST + SSE 流
服务入口	1 个	Nginx 路由到 2 个
状态存储	PostgreSQL	Redis + PostgreSQL
面试讲法	我做了一个 Django 项目	我把AI编排层拆成了独立微服务

四. 拆分方案: 文件级迁移清单

这是最硬核的部分 -- 不是概念上拆, 是一份每个文件放哪里的清单. 照着搬就行.

4.1 新建编排服务项目结构

```
ai-orchestration-service/ # 新建这个目录 (放项目根目录, 和 backend/ 平级) |-- app/ |-- __init__.py |-- main.py # FastAPI 入口 |-- config.py # 配置 (LLM API Key, Redis地址, DB地址) |-- routes/ |-- __init__.py |-- orchestrate.py # POST /execute + GET /status SSE |-- health.py # GET /health | |-- harness/ # 从 backend/core/agents/harness/ 搬来 |-- __init__.py |-- state.py # AgentState (路径修正: 去掉 core. 前缀) |-- workflow.py # build_workflow, run_workflow_stream | |-- agents/ # 从 backend/core/agents/ 搬来 |-- __init__.py |-- base_agent.py |-- plan_agent.py |-- testcase_gen_agent.py |-- data_factory_agent.py |-- execution_agent.py |-- evaluator_agent.py |-- knowledge_agent.py | |-- self_healing/ # 从 self_healing/ 搬来 |-- tools/ # 从 backend/core/tools/ 搬来 (9文件) |-- mcp/ # 从 backend/core/mcp/ 搬来 (4文件) |-- llm/ # 从 backend/core/models/ 搬来 |-- router.py |-- prompts/ | |-- dependencies/ |-- db.py # Redis + PostgreSQL 连接管理 |-- requirements.txt # FastAPI, uvicorn, redis, httpx, langgraph... |-- Dockerfile '-- README.md
```

4.2 核心迁移对照表 (逐文件)

#	当前文件 (backend/)	搬到 (ai-orchestration-service/)	注意事项
1	core/agents/harness/state.py	app/harness/state.py	直接复制, 无改动
2	core/agents/harness/workflow.py	app/harness/workflow.py	改 import: core.agents.xxx -> app.agents.xxx
3	core/agents/base_agent.py	app/agents/base_agent.py	改 import: core.models.router -> app.llm.router
4	core/agents/plan_agent.py	app/agents/plan_agent.py	同上, 改 import 路径
5	core/agents/testcase_gen_agent.py	app/agents/testcase_gen_agent.py	同上
6	core/agents/data_factory_agent.py	app/agents/data_factory_agent.py	同上
7	core/agents/execution_agent.py	app/agents/execution_agent.py	同上
8	core/agents/evaluator_agent.py	app/agents/evaluator_agent.py	同上
9	core/agents/knowledge_agent.py	app/agents/knowledge_agent.py	同上
10	core/agents/self_healing/engine.py	app/self_healing/engine.py	改 import 路径
11	core/tools/*.py (全部9个文件)	app/tools/*.py	改 import: core.xxx -> app.xxx
12	core/mcp/*.py (全部4个文件)	app/mcp/*.py	改 import: core.xxx -> app.xxx
13	core/models/router.py	app/llm/router.py	改 import 路径
14	core/models/prompts/*.py	app/llm/prompts/*.py	直接复制

4.3 唯一需要新写的代码: FastAPI 路由

上面搬的代码都只需要改 import 路径, 不用改任何逻辑. 唯一要新写的是两个 FastAPI 路由文件.

app/routes/orchestrate.py (编排服务核心接口)

```
# app/routes/orchestrate.py
from fastapi import APIRouter
from fastapi.responses import StreamingResponse
from pydantic import BaseModel
from app.harness.workflow import build_default_workflow, run_workflow_stream
from app.harness.state import AgentState
import uuid, json

router = APIRouter()

class OrchestrateRequest(BaseModel):
    user_request: str # 用户需求描述
    user_id: int = 1
    knowledge_base_id: str = None
    project_id: str = None

class OrchestrateResponse(BaseModel):
    task_id: str
    status: str # "accepted"

@router.post("/execute", response_model=OrchestrateResponse)
async def execute_orchestration(req: OrchestrateRequest):
    task_id = str(uuid.uuid4()) # TODO: 异步执行 (Celery / BackgroundTasks)
    return OrchestrateResponse(task_id=task_id, status="accepted")

@router.get("/execute/{task_id}/stream")
async def stream_progress(task_id: str):
    state = AgentState(user_request="...", plan=[], ...)
    workflow = build_default_workflow()
    for event in run_workflow_stream(workflow, state):
        yield f"data: {json.dumps(event)}\n\n"
    yield "data: [DONE]\n\n"
    return StreamingResponse(event_generator(), media_type="text/event-stream", headers={"X-Accel-Buffering": "no"})

@router.get("/health")
async def health():
    return {"status": "ok", "agents": ["plan", "generator", "data_factory", "execution", "evaluator"]}
```

app/main.py (FastAPI 入口)

```
# app/main.py
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from app.routes import orchestrate, health

app = FastAPI(title="AI Agent Orchestration Service", version="1.0")
app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_methods=["*"], allow_headers=["*"])
app.include_router(orchestrate.router, prefix="/api/orchestrate", tags=["Orchestration"])
app.include_router(health.router, prefix="/api", tags=["Health"])
```


五. Step by Step: 7天执行清单

D1 -- 环境就绪

- 确认 ECS 机器 (2核8G) 运行正常, SSH 可以登录
- 确认同事账号 (高级版, 9000积分) 可以正常对话
- 拉取项目最新代码到 ECS
- 跑一遍 docker-compose up, 确认当前 Django 项目能正常启动
- 记录当前端口: Django 8000, PostgreSQL 5432, Nginx 80

D2 -- 创建编排服务骨架

- 在项目根目录创建 ai-orchestration-service/ 目录
- 初始化 app/main.py, app/config.py, requirements.txt
- 安装依赖: FastAPI, uvicorn, httpx, redis, pydantic
- 在 ECS 上启动一个裸 FastAPI 服务, 确认 /health 可访问
- 产出: 目录结构就绪, /health 返回 200

D3 -- 文件迁移: Agent 核心层

- 搬 harness/state.py -> app/harness/state.py (无需改动)
- 搬 harness/workflow.py -> app/harness/workflow.py (改 import)
- 搬 base_agent.py + 6个 Agent 文件 -> app/agents/
- 改所有 import 路径: core.agents.xxx -> app.agents.xxx
- 搬 self_healing/engine.py -> app/self_healing/
- 在 Python 里手动 import 验证: from app.agents.base_agent import BaseAgent

D4 -- 文件迁移: 工具层 + MCP + LLM

- 搬 core/tools/ 全部 9 个文件 -> app/tools/
- 搬 core/mcp/ 全部 4 个文件 -> app/mcp/
- 搬 core/models/router.py -> app/llm/router.py
- 搬 core/models/prompts/ -> app/llm/prompts/
- 全局搜索 core., 全部替换为 app. (只改 import 路径, 不改任何逻辑)

D5 -- 写路由 + 测试

- 写 app/routes/orchestrate.py (POST /execute + GET /stream)
- 写 app/routes/health.py (GET /health)
- 更新 app/main.py, 注册路由
- 本地跑 uvicorn app.main:app --reload, 确认 Swagger 文档可访问
- 用 curl 手动测: POST /api/orchestrate/execute -> 返回 task_id

D6 -- docker-compose 编排

- 写 ai-orchestration-service/Dockerfile
- 更新项目根目录 docker-compose.yml, 新增编排服务容器
- 添加 Redis 容器
- 配置 Nginx 反向代理规则: /api/* -> Django, /orchestrate/* -> FastAPI
- docker-compose up -d, 确认所有服务启动

D7 -- 全链路联调

- 从 Django views.py 发 HTTP POST 到编排服务
- 确认编排服务返回 SSE 流式进度
- 端到端: 用户输入需求 -> Django 转发 -> 编排服务执行 -> SSE 回到前端
- 修复所有联调中发现的 bug
- 产出: docker-compose up 后, 全链路跑通

六. Docker Compose 完整配置

```
# docker-compose.yml (项目根目录) version: '3.8' services: # ---- 业务服务 ---- django: build: ./backend container_name: ai-platform-django ports: - "8000:8000" environment: - DATABASE_URL=postgresql://user:pass@postgres:5432/ai_platform - ORCHESTRATION_SERVICE_URL=http://orchestration:8001 # * 新增 - REDIS_URL=redis://redis:6379/0 depends_on: - postgres - redis volumes: - ./backend:/app # ---- AI 编排服务 * 新增 ---- orchestration: build: ./ai-orchestration-service container_name: ai-orchestration ports: - "8001:8001" environment: - DEEPSEEK_API_KEY=${DEEPSEEK_API_KEY} - DATABASE_URL=postgresql://user:pass@postgres:5432/ai_platform - REDIS_URL=redis://redis:6379/0 depends_on: - postgres - redis volumes: - ./ai-orchestration-service:/app command: uvicorn app.main:app --host 0.0.0.0 --port 8001 --workers 2 # ---- 数据库 ---- postgres: image: postgres:15-alpine ports: - "5432:5432" environment: - POSTGRES_DB=ai_platform - POSTGRES_USER=user - POSTGRES_PASSWORD=pass volumes: - pgdata:/var/lib/postgresql/data # ---- 缓存 * 新增 ---- redis: image: redis:7-alpine ports: - "6379:6379" # ---- 网关 ---- nginx: image: nginx:alpine ports: - "80:80" volumes: - ./nginx.conf:/etc/nginx/nginx.conf:ro volumes: pgdata:
```

Nginx 路由配置 (关键部分)

```
# nginx.conf upstream django { server django:8000; } upstream orchestration { server orchestration:8001; } server { listen 80; # 业务 API -> Django location /api/agent/ { proxy_pass http://django; proxy_buffering off; # SSE 需要关闭缓冲 } # AI 编排 -> FastAPI (SSE 流) location /api/orchestrate/ { proxy_pass http://orchestration; proxy_buffering off; # * SSE 关键配置 proxy_cache off; proxy_read_timeout 600s; # Agent 执行可能很慢 } # 前端静态文件 -> Django location / { proxy_pass http://django; } }
```

七. Django 侧改动: 从直接调用到 HTTP 请求

改动范围很小 -- 只改一个文件: agent_gateway/views.py.

不需要动任何 Django Model, Serializer, 前端代码. 只把 import Agent 替换为发 HTTP 请求到编排服务.

改之前 (直接 import Agent 在 Django 进程内执行):

```
# agent_gateway/views.py (旧版) from core.agents.harness.workflow import build_default_workflow, run_workflow_stream from
core.agents.testcase_gen_agent import TestCaseGeneratorAgent def execute_workflow_view(request): workflow =
build_default_workflow() state = AgentState(user_request=request.POST['requirement'], ...) # Agent 在 Django 进程里直接跑 results
= list(run_workflow_stream(workflow, state)) return Response(results)
```

改之后 (发 HTTP 请求, Agent 在独立服务中执行):

```
# agent_gateway/views.py (新版) import os, httpx, json ORCHESTRATION_URL = os.getenv('ORCHESTRATION_SERVICE_URL',
'http://orchestration:8001') @action(detail=False, methods=['post']) def run_harness(self, request): # 1. 发起编排任务 -> 编排服务
async with httpx.AsyncClient() as client: resp = await client.post( f'{ORCHESTRATION_URL}/api/orchestrate/execute', json={
"user_request": request.data['requirement'], "user_id": request.user.id, "knowledge_base_id": request.data.get('kb_id'), }, timeout=30
) data = resp.json() task_id = data['task_id'] # 2. 建立 SSE 连接 -> 转发给前端 (透明代理) sse_resp = await client.get(
f'{ORCHESTRATION_URL}/api/orchestrate/execute/{task_id}/stream', timeout=None # 不限时 ) # 3. 流式返回给前端 (前端无感知)
response = StreamingHttpResponse( sse_resp.aiter_bytes(), content_type='text/event-stream' ) response['X-Accel-Buffering'] = 'no'
return response
```

[改动总结] Django 不再直接运行 Agent, 只负责接收用户请求并转发给编排服务. 原来 import Agent 的那几行删掉, 换成 HTTP 调用.

八. 第二周: 加固与交付物

天	任务	产出
D8	Redis 状态缓存: Agent运行时状态写入Redis (5分钟TTL)	状态外置, 服务重启不丢任务
D9	错误处理: HTTP超时重试3次, 降级回退, 异常日志	韧性保障
D10	压测: wrk -t4 -c50 -d30s 跑编排服务 记录吞吐量 + P50/P99延迟	压测报告.md
D11	结构化日志: JSON格式, 带 trace_id 可选 Loki 聚合	可观测性
D12	deploy.sh + 健康检查 从零环境到全服务启动 < 2分钟	交付脚本
D13	架构决策记录(ADR): 为什么拆 / 为什么不用K8s / 为什么FastAPI	架构文档
D14	Swagger截图 + 演示录屏 + 简历更新	完整交付物

九. 面试话术对照 (拆完后怎么讲)

Q: 你为什么要做微服务拆分?

A: 不是单体跑不动, 是提前验证企业场景. AI编排层的运算密度和业务层完全不同 -- 业务API是短连接, Agent调用LLM可能要30秒. 同一个Worker里两种任务争资源. 拆开后编排服务可以独立扩缩容, 业务层完全不受 Agent 执行时长影响.

Q: 拆的过程中遇到什么困难?

A: 最大的坑是 import 路径全面重构. 原来 core.agents.xxx 有 200+ 处引用, 搬过去全部改成 app.agents.xxx, 还要保证单元测试全过. 第二个坑是 SSE 流式在 Nginx 反向代理下的缓冲问题, 必须设 proxy_buffering off.

Q: 为什么不用 K8s 或 Istio?

A: 当前就两个服务, Docker Compose + Nginx 反向代理完全够用. 等后续服务数量超过 5 个, 需要灰度发布或弹性伸缩时再引入 K8s. 架构选型的核心不是技术多新, 是匹配当前复杂度.

Q: 状态管理怎么处理的?

A: Redis 做运行时缓存 (Agent 执行中状态, 5分钟TTL), PostgreSQL 做最终持久化. 不用 Django Session 管 Agent 状态, 因为编排服务需要独立感知状态变化.

Q: 如果编排服务挂了怎么办?

A: 三重保障: 1. 编排服务重启后从 PostgreSQL 恢复未完成任务 2. Django 侧设置 3 次自动重试 3. Nginx health check 自动摘除不可用实例

附录: 完整文件结构对比

拆之前 (backend/ 关键部分)

```
backend/ |-- agent_gateway/views.py # 直接 import Agent, 改这里 |-- agent_gateway/tasks.py # Celery 任务, 改这里 |-- core/agents/
# * 搬走 |-- core/tools/ # * 搬走 |-- core/mcp/ # * 搬走 |-- core/models/router.py # * 搬走 '-- (其他 Django 文件不动)
```

拆之后 (新增 ai-orchestration-service/)

```
ai-orchestration-service/ |-- app/ | |-- main.py # FastAPI 入口 (新写) | |-- config.py # 配置 (新写) | |-- routes/ | | |-- orchestrate.py #
REST + SSE 路由 (新写) | | |-- health.py # 健康检查 (新写) | |-- harness/ # 搬来的, 改 import | |-- agents/ # 搬来的 7 个文件, 改 import |
|-- self_healing/ # 搬来的 | |-- tools/ # 搬来的 9 个文件 | |-- mcp/ # 搬来的 4 个文件 | '-- llm/ # 搬来的 router.py + prompts |--
requirements.txt |-- Dockerfile '-- README.md
```

开始动手. 第一个 docker-compose up 跑通就成功了 80%.